

Finals - Questions & Answers (1-12)

Q1 - What is tokenization? Give an example - show how the sentence "I'm learning NLP in 2025!" would be tokenized.

Answer:

Tokenizing just splits text into pieces - words, numbers, punctuation, whatever.

Take "I'm learning NLP in 2025!". It becomes:

["I", "m", "learning", "NLP", "in", "2025", "!"]

That way the computer can handle each piece on its own, even with weird punctuation mixed in.

Q2 - What is the difference between stemming and lemmatization? Apply both to the words "running" and "better" and explain which preserves more linguistic meaning.

Answer:

Stemming strips suffixes based on rules. Lemmatization gives you the original form (lemma) - the actual word you'd find in a dictionary.

For example: "running"

- Stemming: run
- Lemmatization: run

But the word "better":

- Stemming: better (remains the same)
- Lemmatization: good

Lemmatization keeps more meaning because you get the real word, not just a chopped-up version.

Q3 - What does TF-IDF stand for? Explain in plain language why the word "the" scores almost zero in TF-IDF, while the word "photosynthesis" would score high.

Answer:

TF-IDF means Term Frequency times Inverse Document Frequency.

TF-IDF is about finding words that actually matter.

"The" appears almost everywhere - in news, books, emails. So it doesn't help us understand what makes a document special.

"Photosynthesis" is rare. It mostly shows up in science texts. If a document uses it a lot, it's probably about biology.

The idea is simple: look for words that are

1. Common in THIS document
2. Rare in OTHER documents

Those are the words that tell you what the document is really about.

Q4 - What is a sentence embedding? How is it fundamentally different from one-hot encoding? Give one advantage embeddings have that one-hot vectors don't.

Answer:

A sentence embedding is basically one vector that captures what the whole sentence means. Similar sentences end up close together in space.

One-hot encoding is different - it gives every word its own slot, a vector full of zeros with a single 1. So "cat" and "kitten" are totally unrelated, even though they mean almost the same thing.

The big advantage with embeddings is you can actually check if two sentences mean the same thing - like with cosine similarity. With one-hot, you just can't do that.

Q5 - Explain cosine similarity in plain language. If two document vectors point in almost the same direction, what does that tell us about the documents they represent?

Answer:

Cosine similarity calculates the angle between two vectors.

If two document vectors point in almost the same direction, it usually means the documents are similar in meaning, even if they don't use exactly the same words.

That's why cosine similarity is often better than Euclidean distance for embeddings: cosine focuses on direction, while Euclidean distance is more affected by vector length.

Q6 - Why can't a regular SQL query like `WHERE description LIKE '%pizza%'` find semantically similar documents? What does a vector index solve that SQL can't?

Answer:

A `LIKE '%pizza%'` query only does exact substring matching. If you search for "pizza", you will only get rows that literally contain the word "pizza" in the text.

Expressions like "Italian food" or "pasta dishes" will be missed, even though they describe related content.

With a vector index, we first turn each document into an embedding and do a nearest-neighbors search. That way we can retrieve chunks that are close in meaning to the question, not only those that repeat the same word.

Q7 - What problem does RAG solve that a plain LLM (without RAG) cannot? Give a concrete example of when you would choose RAG over just prompting the LLM directly.

Answer:

RAG gives an LLM actual documents to read, so it answers based on real information instead of guessing.

This prevents hallucinations and lets it use private or recent data that wasn't in its training.

For example: A chatbot answering questions about company policies without RAG might give wrong information from memory. With RAG, it pulls the actual policy document first, so it gives accurate answers with real sources.

Q8 - Describe the 3 main steps of a RAG pipeline in the correct order. Be clear about what happens at ingestion time (when you load documents) vs query time (when a user asks a question).

Answer:

Ingestion time (loading documents):

First, documents are broken into smaller pieces. Each piece becomes an embedding. Then all embeddings get saved in a vector database.

Query time (user asks a question):

The question gets converted to an embedding. The system finds the most similar embeddings from the database. Finally, those matching pieces are sent to the LLM so it can write an answer.

Q9 - What is the difference between a Docker image and a Docker container? Use an analogy to explain.

Answer:

Docker image is like a recipe - it contains all the instructions and ingredients needed to run an application.

A Docker container is what you get when you actually run that recipe - it's the live, running instance. You can start, stop, or delete containers, but the image stays the same.

The same image can launch many containers, just like one recipe can make many cakes.

Q10 - What is the difference between a simple LLM chatbot and an AI agent with tools? Give one concrete example of a "tool" and explain why it makes the agent more capable.

Answer:

a simple chatbot can only write text based on what it learned. It might suggest a meeting time, but it can't actually book it. an AI agent can use tools to do real things.

for example, a calendar tool lets it check availability and create the event automatically.

the difference is that chatbot just talks, while an agent can take action.

Q11 - What is MCP (Model Context Protocol)? What problem does it solve for AI coding assistants like GitHub Copilot? Name two examples of things an MCP server might expose to an AI assistant.

Answer:

MCP is a standard that lets AI assistants discover and use tools from other programs in a consistent way.

This solves the mess of building custom integrations for every tool. An MCP server advertises what it can do, and the assistant can request those actions in a standard format.

Examples:

A database server that lets the AI run queries and fetch data

A web search server that lets it look up information online

Q12 - What are Agent Skills in the context of AI coding assistants? How are they different from just writing instructions in a plain prompt? Show a minimal example of what a skill's .md metadata block might look like.

Answer:

Agent Skills are pre-built instruction packages that the AI can reuse across many projects.

A normal prompt is typed once for one chat. A Skill is saved, organized, and used over and over — so the AI behaves consistently without you having to rewrite everything.

Minimal metadata example:

```
<skill>
  <name>docker-debugger</name>
  <description>
    Helps fix Docker problems.
    Activate when the user talks about containers not starting
    or service failures.
  </description>
  <file>skills/docker-debugger/SKILL.md</file>
</skill>
```